

Universidad CENFOTEC

Estructuras de Datos:

Primer Estudio de Caso

Estudiante:

Pablo Andres Chinchilla Moya

Profesor:

Romario Salas Cerdas

Alajuela, 11 de octubre de 2026

El análisis de cadenas de impresión

Los tokens

Un token es la unidad mínima con significado dentro de una expresión; separar el texto en tokens es la primera etapa del análisis léxico que realizan los compiladores e intérpretes (Aho et al., 2006). En las cadenas de impresión se distinguen tres tipos:

Tipo de token	Qué representa	Ejemplo
Cadena literal	Texto escrito entre comillas, que se imprime tal cual	"Hola "
Variable de cadena	Un nombre cuyo valor es una cadena guardada en memoria	nombre
Operador de concatenación	El símbolo que une dos operandos en una sola cadena	+

Con esa clasificación, la instrucción "Hola " + nombre + "!" se parte en cinco tokens: la cadena literal "Hola ", el operador +, la variable nombre, el operador + y la cadena literal "!".

La partición no puede hacerse simplemente cortando el texto por espacios o por el símbolo +. Dentro de un literal puede haber espacios e incluso un + que es texto y no operador, como en "a + b". Por eso el proceso recorre la cadena carácter por carácter y recuerda en todo momento si está dentro o fuera de unas comillas.

¿Qué revisa el análisis?

Una vez obtenidos los tokens, el análisis cumple dos objetivos. El primero es verificar la estructura: las comillas deben estar cerradas, los operandos (literales y variables) y los operadores deben alternarse, y la expresión no puede empezar ni terminar con un operador. El segundo es evaluar la expresión cuando es válida: cada variable se reemplaza por su valor y todos los operandos se unen, de izquierda a derecha, en la cadena final que se imprime.

Por qué la pila es la estructura adecuada

Una pila es una estructura que sigue el principio LIFO (*Last In, First Out*): el último elemento en entrar es el primero en salir, y solo se puede acceder al elemento que está en el tope (Cormen et al., 2022). Sus operaciones fundamentales son push (apilar un elemento), pop (sacar el elemento del tope), peep (consultar el tope sin sacarlo) y estaVacia (indicar si no hay elementos).

Los requerimientos del análisis de cadenas de impresión coinciden con lo que la pila ofrece:

1. **Se procesa en un solo sentido.** La cadena se lee una sola vez, de izquierda a derecha, y cada token se apila en el orden en que aparece.
2. **Hay elementos que esperan a otros.** Al leer "Hola " todavía no se puede hacer nada con él, porque falta saber qué operador sigue y con qué operando se unirá. La pila conserva esos tokens pendientes sin perder su orden.
3. **Se resuelve primero lo más reciente.** Como la concatenación se resuelve a partir del extremo derecho de la expresión (así lo hace también Lua, como se verá en el apartado 5), el primer token que se necesita es justamente el último que entró. Sacarlo del tope es la operación natural de la pila, sin recorrer ni buscar en el resto de la estructura.
4. **Se necesita revisar antes de actuar.** Para validar la estructura hay que mirar el token siguiente sin sacarlo todavía: ¿es un operando o un operador? Esa es la función de peep.
5. **La longitud de la expresión no se conoce de antemano.** Una cadena de impresión puede unir dos operandos o cincuenta, por lo que conviene una

pila dinámica, que crece según se necesite, en lugar de una estática de tamaño fijo, que obligaría a decidir un máximo y a manejar el caso de pila llena.

Es cierto que, si la expresión solo tuviera un operador y ningún paréntesis, también podría resolverse con una variable acumuladora. Sin embargo, la pila permite validar la estructura con una regla uniforme, deja visibles los pasos intermedios y se extiende sin cambiar el diseño a casos más ricos, como paréntesis o varios tipos de operadores.

Otras estructuras encajan peor. Una cola entregaría primero el token más antiguo, mientras que el análisis necesita primero el más reciente. Una lista o un arreglo permitirían acceder a cualquier posición, algo innecesario aquí que además elimina la protección que da el acceso exclusivo por el tope.

Seguridad, orden y eficiencia

Seguridad

- **Acceso restringido.** El resto del programa solo puede usar push, pop y peep, por lo que ningún elemento intermedio puede modificarse por accidente.
- **Errores controlados.** Sacar o consultar el tope de una pila vacía (subdesbordamiento) se detecta con estaVacia y se reporta con un mensaje, en lugar de provocar una falla del programa; en una pila estática ocurre lo mismo al apilar cuando está llena (desbordamiento).
- **El texto se trata como dato.** Analizar la cadena y descomponerla en tokens conocidos antes de imprimirla evita que texto externo se interprete como instrucción. Un ejemplo grave es usar datos externos como cadena de formato en funciones como printf del lenguaje C, lo que puede permitir a un atacante leer o escribir memoria; esta debilidad está catalogada como CWE-134 (MITRE, s. f.).

Orden

- **Orden determinista.** Una misma expresión siempre produce la misma secuencia de operaciones push, peep y pop; el resultado no depende de nada más que de los tokens.
- **Orden de los operandos.** La concatenación no es conmutativa: "Hola " + "Pablo" no es igual a "Pablo" + "Hola ". Como los tokens salen de la pila en orden inverso al que entraron, el análisis debe ir colocando cada valor por delante de lo que ya tiene acumulado; así la cadena final conserva el orden original de izquierda a derecha.
- **Estado final verificable.** Si la expresión es válida, la pila termina vacía y hay un resultado completo; si no lo es, el punto exacto en el que el análisis se detiene indica cuál es el error.

Eficiencia

- Cada operación de la pila (push, pop, peep, estaVacia) se ejecuta en tiempo constante, O(1): no depende de cuántos elementos haya almacenados. En una implementación dinámica con ArrayList, apilar y desapilar en el extremo final tiene ese mismo costo (amortizado en el caso de apilar).
- El análisis apila cada token una vez y lo saca una vez, por lo que el recorrido completo es proporcional a la cantidad de tokens, O(n).
- Unir cadenas de dos en dos con + crea una cadena nueva en cada paso, lo que puede volverse costoso con textos muy largos. Por eso las implementaciones reales evitan repetir ese trabajo: el intérprete de Lua, por ejemplo, calcula el largo total de todas las cadenas que va a unir y crea el resultado una sola vez (Lua.org, s. f.).

Ejemplo concreto: el intérprete de Lua

Contexto

Los intérpretes y compiladores forman parte del software de sistema, y una de sus tareas es analizar y evaluar expresiones como las cadenas de impresión. Lua es un lenguaje de programación pequeño, desarrollado en la PUC-Rio (Brasil) y

pensado para integrarse dentro de otros programas; se usa, por ejemplo, para escribir scripts dentro de Redis y para configurar el editor Neovim.

Para que un programa escrito en C pueda comunicarse con código Lua, el intérprete ofrece una pila virtual: los valores que viajan entre C y Lua se apilan y desapilan con las funciones de su interfaz de programación (Ierusalimschy et al., 2023; Ierusalimschy, 2016).

Cómo concatena Lua

En Lua, el operador de concatenación es .. y es asociativo por la derecha (Ierusalimschy et al., 2023); en "Hola " .. nombre .. "!" la unión se resuelve entonces a partir del extremo derecho, que es el tope de la pila. En el código fuente del intérprete, la función luaV_concat se documenta como la operación principal de concatenación: trabaja sobre los valores que están en la pila, desde la posición top - total hasta top - 1; toma de una vez todas las cadenas consecutivas que pueda, calcula el largo total, crea una única cadena resultado, saca de la pila las cadenas usadas y deja el resultado en el tope (Lua.org, s. f.). Esa misma operación está disponible para los programas en C con la función lua_concat(L, n), que une los n valores del tope de la pila (Ierusalimschy et al., 2023).

Aclaración: El contexto y la investigación del ejemplo, así cómo el resto del documento, fueron realizados por mi persona, sin embargo, el código utilizado en las imágenes ilustradoras que acompañan al ejemplo fue construido con la ayuda de la herramienta de Inteligencia Artificial Claude. Esto se debe a que no se cuenta con el conocimiento necesario del lenguaje C y Lua para confeccionar el código manualmente. No obstante, el ejemplo busca presentar un caso de implementación del análisis de cadenas de impresión, y no se deseaba que la limitación del conocimiento imposibilitara el uso del ejemplo investigado. El código creado para el ejemplo también será incluido en la rama estudios-de-caso del repositorio del curso.

Resultados observados

Para comprobar este comportamiento se creó un programa en C, compilado contra la biblioteca de Lua 5.4.6, que apila tres cadenas ("Hola ", "Pablo" y "!"),

muestra el contenido de la pila, llama a lua_concat(L, 3) y vuelve a mostrar la pila. El resultado se resume en la tabla y se muestra en la Figura 1:

Momento	Nivel 3 (tope)	Nivel 2	Nivel 1 (base)	Elementos
Antes de lua_concat(L, 3)	"!"	"Pablo"	"Hola "	3
Después de lua_concat(L, 3)	–	–	"Hola Pablo!"	1

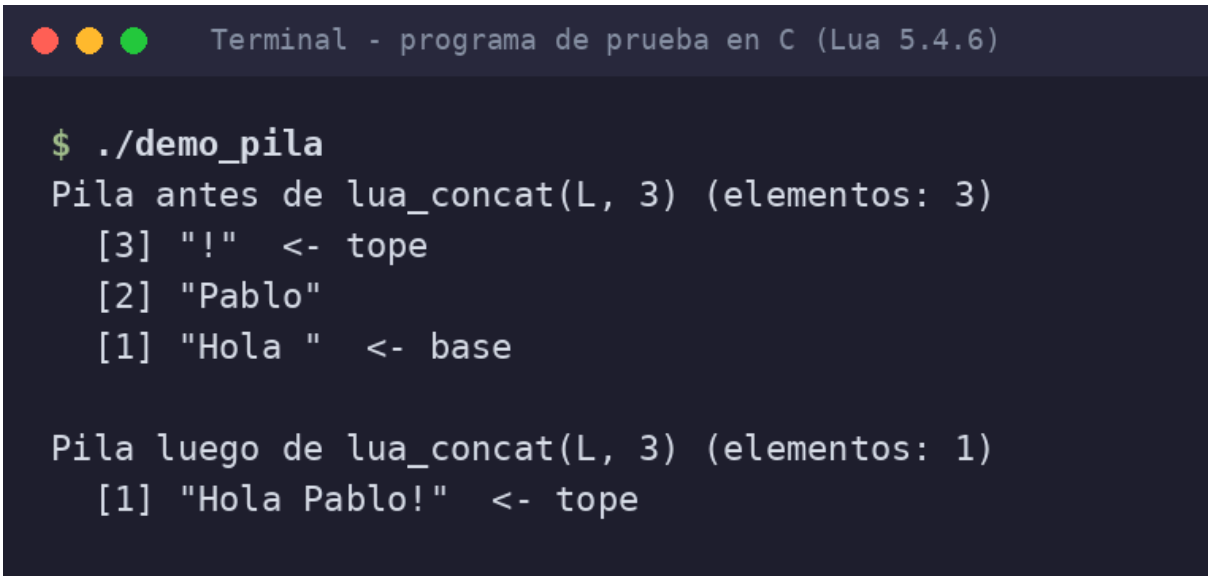


Figura 1. Salida del programa de prueba en C: contenido de la pila de Lua antes y después de llamar a lua_concat(L, 3).

Además se probó cómo reacciona Lua ante cadenas de impresión mal escritas, que el intérprete analiza antes de ejecutarlas (Figura 2):

Instrucción	Respuesta de Lua
print("Hola " .. "mundo")	Imprime Hola mundo
print("Hola " ..)	Error de sintaxis: unexpected symbol near ')'
print("Hola)	Error de sintaxis: unfinished string near <eof>
print("Hola " .. x) (con x sin valor)	Error en ejecución: attempt to concatenate a nil value (global 'x')

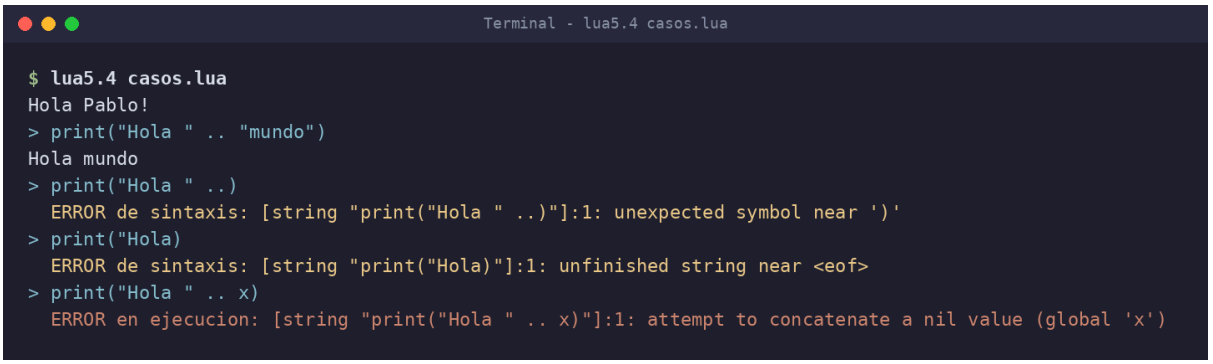


Figura 2. Respuesta del intérprete de Lua ante una cadena de impresión válida y tres inválidas.

Estos tres errores equivalen a tres de las validaciones del análisis propuesto en este documento: operador sin operando, comillas sin cerrar y variable sin valor.

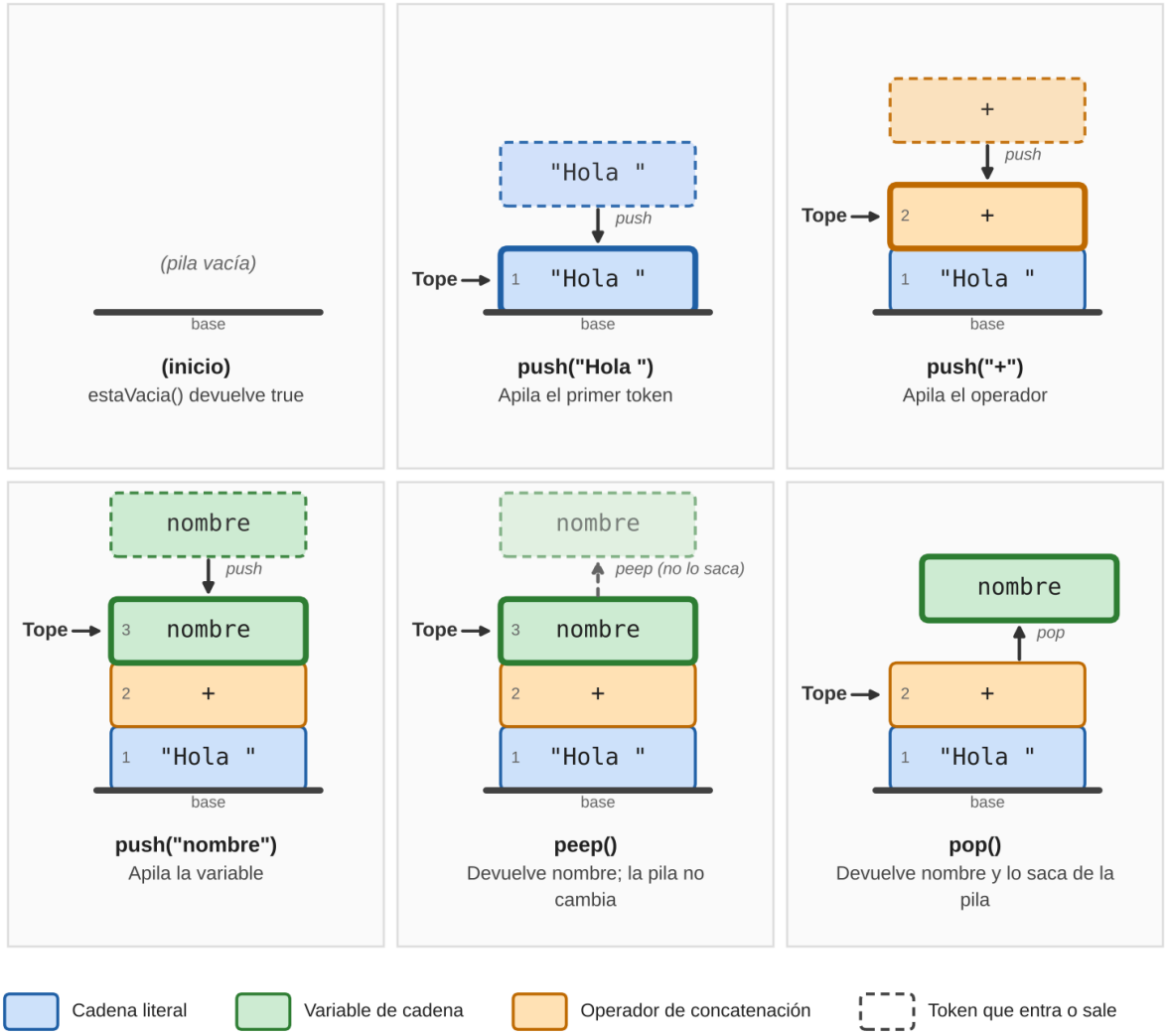
Relación con este estudio

Lua y el programa de este estudio comparten la misma idea: los operandos se apilan en el orden en que aparecen y la concatenación consume elementos desde el tope, reemplazándolos por un único resultado. El programa en Java hace explícito ese proceso con una pila de tokens.

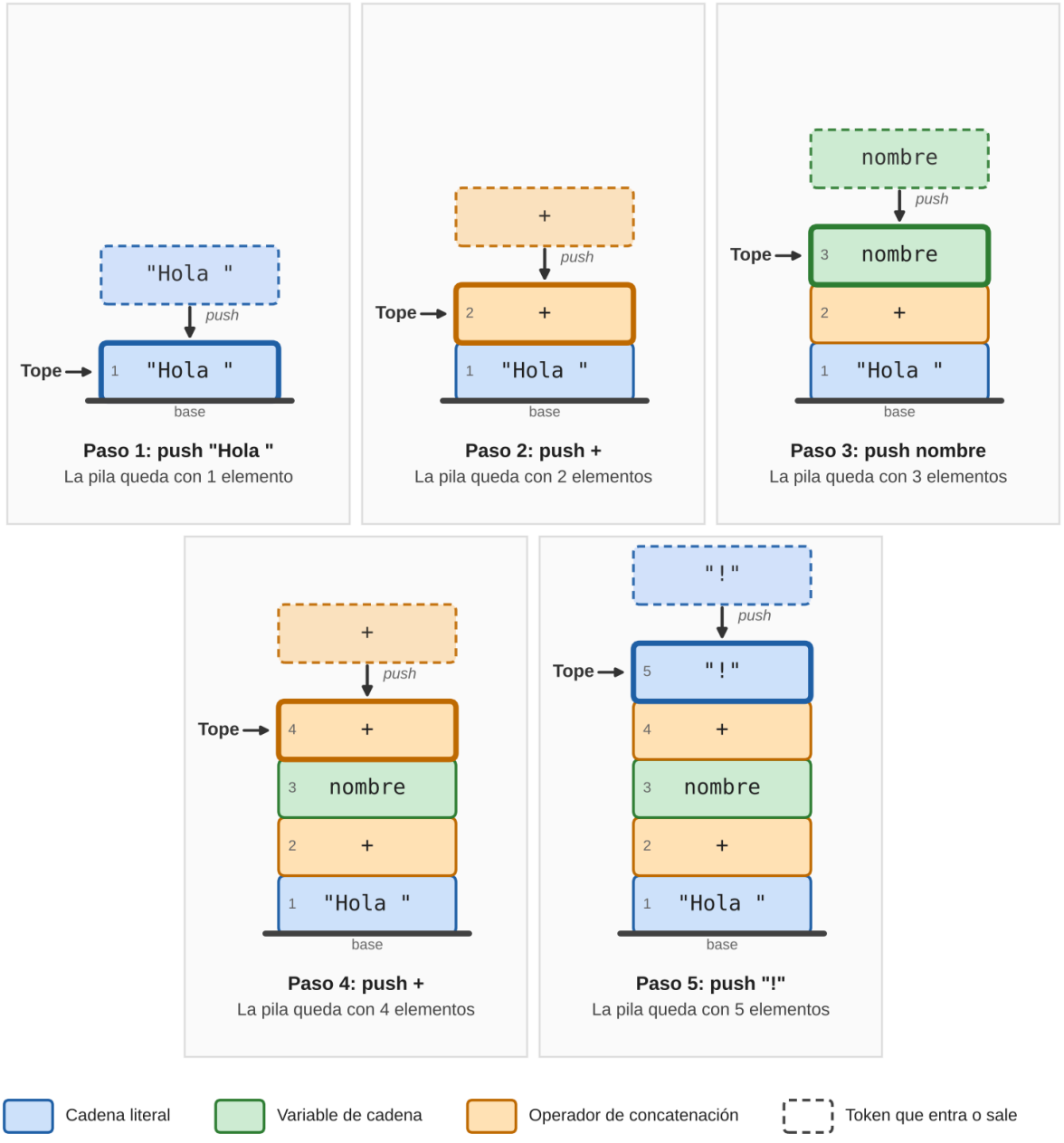
Representación gráfica de las operaciones de la pila

Las siguientes tablas representan la pila como una columna de casillas, igual que se haría con las celdas de una hoja de cálculo. El nivel 1 es la base y el elemento con el nivel más alto ocupado es el tope (en negrita). El ejemplo usa la expresión "Hola " + nombre + "!", con la variable nombre valiendo "Pablo".

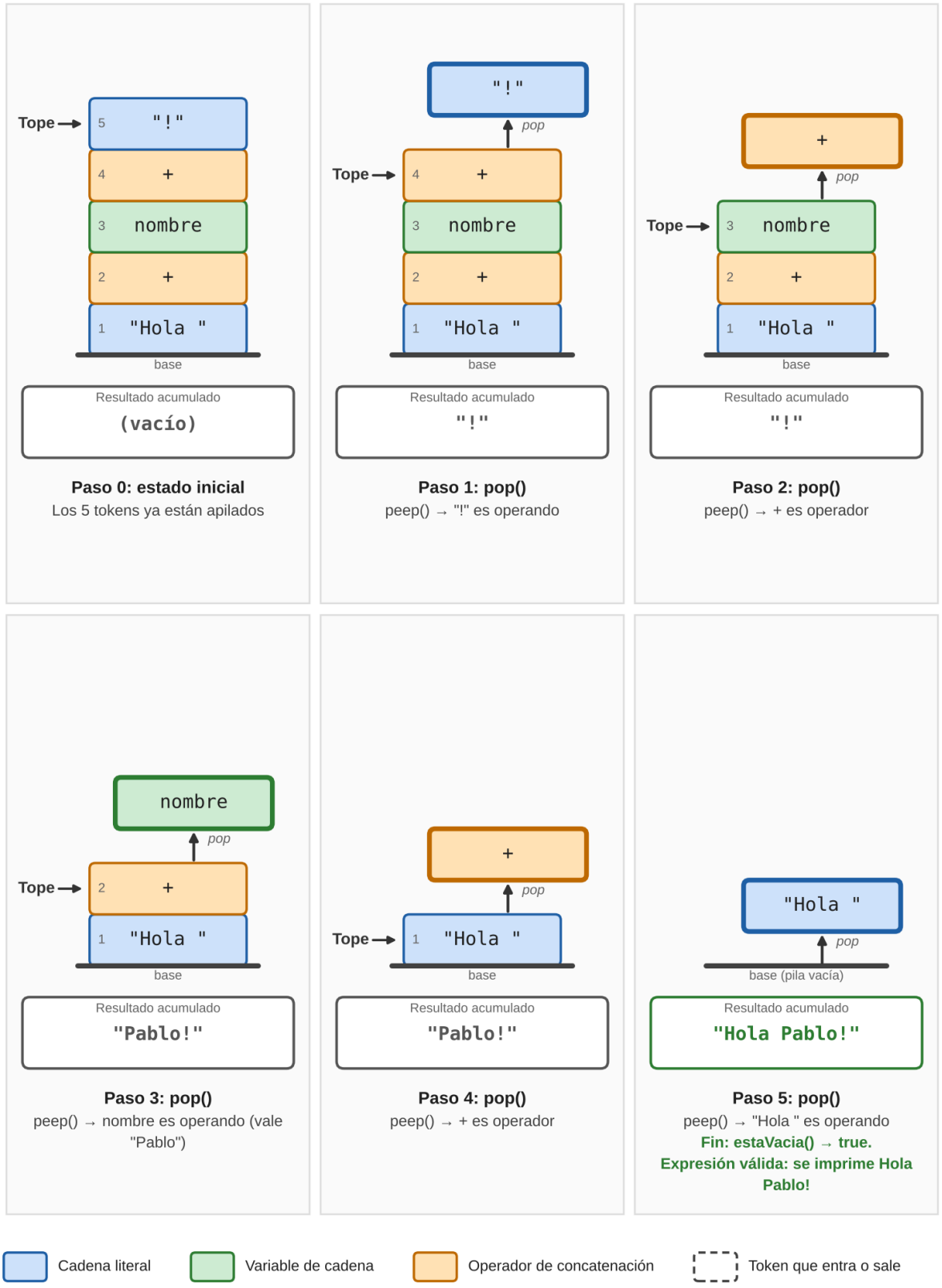
Operaciones fundamentales



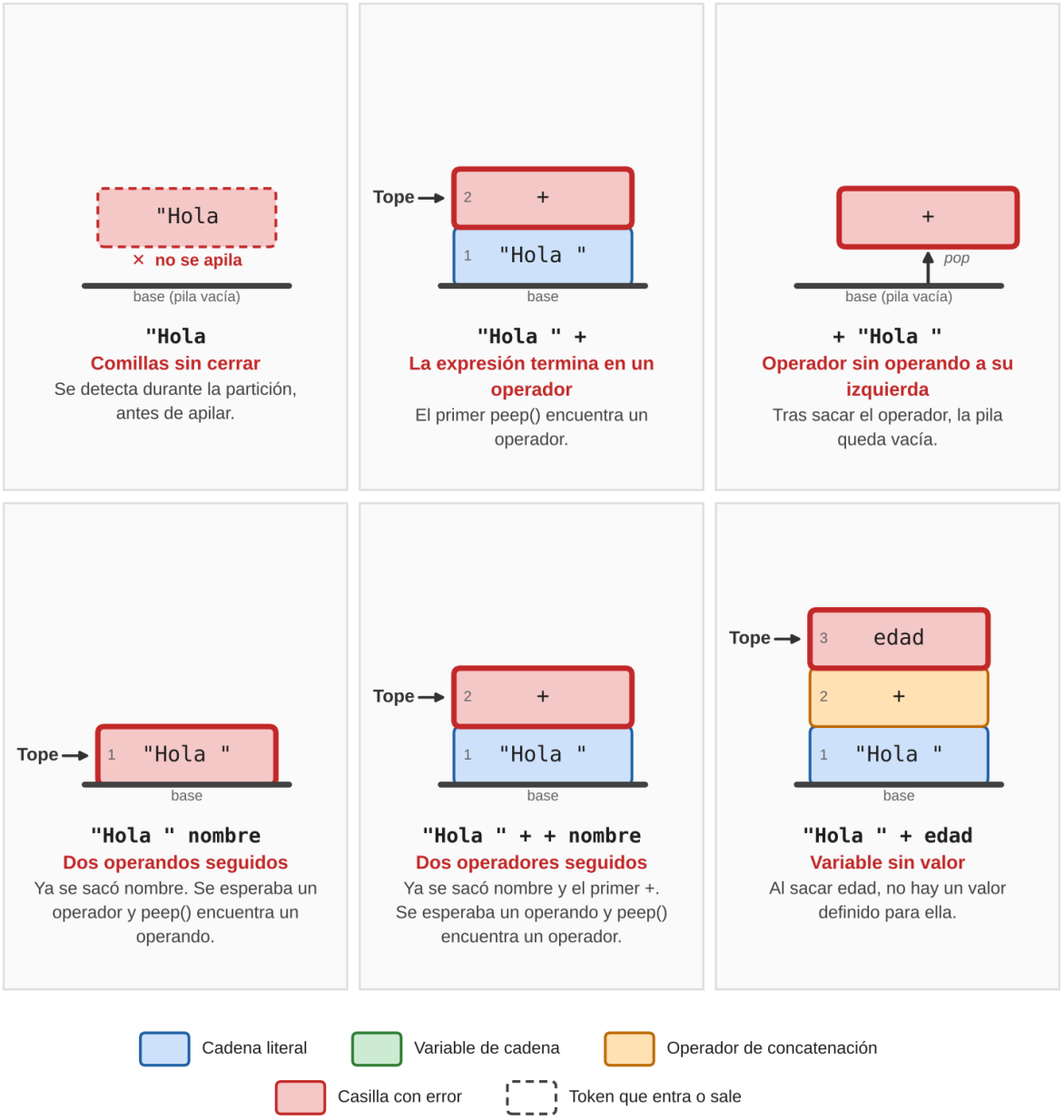
Fase 1: apilado de los tokens



Fase 2: análisis y evaluación desde el tope



Detección de errores



Diseño de la implementación en Java

La implementación que acompaña este documento se organiza así:

- **Token (clase nodo):** guarda el texto del token y su tipo (cadena literal, variable de cadena u operador de concatenación), con sus métodos de consulta y modificación.

- **PilaTokens (clase pila):** pila dinámica de objetos Token con las operaciones push, pop, peep y estaVacia, que avisa cuando se intenta sacar o consultar el tope de una pila vacía.
- **Main:** contiene la rutina main(), que invoca el menu(), y los métodos que parten la cadena en tokens, los apilan y ejecutan el análisis descrito en el apartado 6.

El código fuente completo está disponible en el siguiente repositorio de GitHub: https://github.com/pabloch01/SOFT-10-Estructuras-de-Datos.git

Conclusiones y recomendaciones

Conclusiones

- La pila es adecuada para analizar cadenas de impresión porque los tokens se procesan en un solo sentido, algunos deben esperar a otros y lo que se resuelve primero es siempre lo más reciente, que está en el tope.
- Aporta seguridad (acceso controlado y errores detectados en lugar de fallas), orden (resultado determinista y verificable al final) y eficiencia (operaciones en tiempo constante y un recorrido proporcional a la cantidad de tokens).
- El caso de Lua demuestra que la idea no es solo un ejercicio académico: un intérprete real apila los operandos y concatena desde el tope, y además detecta errores en cadenas mal formadas antes de ejecutarlas.

Recomendaciones

- Ampliar el análisis para contemplar comillas escapadas dentro de un literal y paréntesis. Con paréntesis la pila deja de ser opcional y su profundidad pasa a depender de la expresión, lo que justifica todavía más una implementación dinámica.
- Construir la cadena final en una sola pasada (por ejemplo, con StringBuilder) en lugar de unir cadenas de dos en dos, para evitar copias innecesarias en textos largos.
- Validar y descomponer siempre las cadenas antes de imprimirlas o usarlas como formato, sin tratar texto externo como instrucciones.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.ª ed.). Pearson Education.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Ierusalimschy, R. (2016). *Programming in Lua* (4.ª ed.). Lua.org.

Ierusalimschy, R., de Figueiredo, L. H., & Celes, W. (2023). *Lua 5.4 reference manual*. Lua.org, PUC-Rio. https://www.lua.org/manual/5.4/

Lua.org, PUC-Rio. (s. f.). *lvm.c* [Código fuente del intérprete de Lua]. GitHub. https://github.com/lua/lua/blob/master/lvm.c

MITRE. (s. f.). *CWE-134: Use of externally-controlled format string*. Common Weakness Enumeration. https://cwe.mitre.org/data/definitions/134.html